AGENDEI · MARCOU, CONFIRMOU.

Guia de preparação para a banca

Análise do sistema, perguntas prováveis e respostas sugeridas

Material de estudo baseado na implementação atual do projeto acadêmico Agendei.

Projeto desenvolvido em PHP, HTML, CSS, JavaScript e banco de dados relacional.

1. Visão geral do projeto

> **Apresentação em 30 segundos:**
> O Agendei é um sistema web para gerenciamento de agendamentos. O cliente escolhe serviço, unidade, profissional, data e horário. O sistema calcula a disponibilidade com base no expediente, duração do serviço, bloqueios e reservas existentes. O administrador gerencia usuários, profissionais, serviços e relatórios. A aplicação foi desenvolvida em PHP, HTML, CSS e JavaScript, com PDO para comunicação com MySQL/MariaDB ou PostgreSQL.

Fluxo técnico

```
Navegador
    ↓
Página PHP ou endpoint da API
    ↓
Autenticação e contexto do estabelecimento
    ↓
Model responsável pela regra de negócio
    ↓
PDO com consulta preparada
    ↓
Banco de dados
```

Perfis do sistema

Perfil	Responsabilidade	Escopo acadêmico
Cliente	Realiza, consulta e cancela os próprios agendamentos.	Usuário comum.
Administrador local	Gerencia o estabelecimento, equipe, serviços, agenda e relatórios.	Usuário master.
Profissional	Consulta sua agenda, define horários e bloqueios.	Recurso adicional.
Master global	Administra empresas, planos, segurança e saúde da plataforma.	Recurso adicional.

2. Perguntas prováveis e respostas

1. Qual problema o sistema resolve?

Centraliza os agendamentos de um estabelecimento, reduz o controle manual por papel ou mensagens e permite que o cliente consulte horários realmente disponíveis, faça reservas e acompanhe seu histórico.

2. Quem utiliza o sistema?

Cliente, administrador local, profissional e master global. Para a especificação da faculdade, cliente representa o usuário comum e administrador local representa o usuário master.

3. Quais tecnologias foram utilizadas?

PHP 8 no servidor, HTML e CSS na interface, JavaScript para interações dinâmicas e PDO para comunicação com o banco. O banco padrão é MySQL/MariaDB, com suporte adicional a PostgreSQL/Supabase. Não foram utilizados frameworks como Laravel ou React.

4. O sistema utiliza MVC?

Possui separação em camadas inspirada em MVC, mas não é um MVC formal. As páginas PHP exercem o papel de controlador e view, enquanto os models concentram regras de negócio e acesso ao banco.

5. Como o banco interage com o sistema?

A classe `Database` cria uma conexão PDO. As páginas chamam os models, e os models executam consultas preparadas. Em um agendamento, por exemplo, o servidor recebe os identificadores, consulta as entidades, valida a disponibilidade, executa o `INSERT` e confirma a transação.

6. Por que utilizar PDO?

PDO oferece uma interface padronizada de conexão, consultas preparadas, erros por exceções e suporte a mais de um banco. As consultas preparadas reduzem o risco de SQL Injection ao separar os dados do comando SQL.

7. Quais são as principais tabelas?

`usuarios`, `clientes`, `profissionais`, `servicos`, `profissional_servico`, `horarios_profissionais`, `bloqueios_agenda`, `agendamentos`, `filiais` e `logs_autenticacao`. O arquivo MySQL atual cria 26 tabelas.

8. Por que existe a tabela profissional_servico?

Porque um profissional pode executar vários serviços e um serviço pode ser executado por vários profissionais. É um relacionamento muitos-para-muitos, resolvido por uma tabela associativa.

9. Como o sistema identifica a empresa?

Cada estabelecimento possui um `id_estabelecimento` e um endereço exclusivo, chamado `slug`. A classe `Contexto` resolve a empresa pela URL ou pela sessão. Depois do login, a empresa da sessão não pode ser trocada simplesmente alterando a URL.

10. Como os dados de empresas diferentes ficam separados?

As tabelas operacionais carregam `id_estabelecimento`. As consultas filtram por esse identificador e existem chaves estrangeiras compostas para impedir associações entre registros de empresas diferentes. Esse modelo é multi-tenancy com banco compartilhado.

11. Como funciona o processo de agendamento?

O cliente escolhe serviço, filial, profissional, data e horário. A interface consulta os horários livres por uma API interna. Na confirmação, o servidor revalida tudo e grava a reserva em uma transação.

12. Como os horários livres são calculados?

O sistema consulta a duração do serviço, expediente do profissional, intervalo dos horários, reservas existentes, bloqueios, antecedência mínima e máxima. Em seguida, percorre o expediente e elimina os intervalos com conflito.

13. Por que a duração do serviço é importante?

Porque não basta conferir o horário inicial. Um serviço iniciado às 10h e com duração de 60 minutos ocupa todo o período até 11h. A verificação compara a sobreposição entre os intervalos.

14. O que acontece se duas pessoas tentarem reservar o mesmo horário?

O sistema inicia uma transação e revalida o conflito com `SELECT ... FOR UPDATE`. Se detectar ocupação, executa `rollback`; se estiver livre, insere e executa `commit`. A ideia principal é não confiar apenas no horário que apareceu anteriormente na tela.

15. Para que serve uma transação?

Para tratar operações relacionadas como uma unidade: ou todas são concluídas ou nenhuma permanece parcialmente gravada. `beginTransaction()` inicia, `commit()` confirma e `rollBack()` desfaz.

16. O usuário pode alterar o preço pelo navegador?

Ele pode tentar manipular o formulário, mas o servidor não confia nesse valor. Preço e duração são novamente consultados no banco, e a disponibilidade também é recalculada antes da gravação.

17. Por que validar no JavaScript e novamente no PHP?

O JavaScript melhora a experiência e mostra erros rapidamente, mas pode ser desativado ou alterado. O PHP executa a validação definitiva. Frase para memorizar: **“JavaScript ajuda o usuário; PHP protege o sistema.”**

18. Como funciona o login?

O sistema localiza a conta por login ou e-mail, verifica o hash da senha, o estado da conta e do estabelecimento e, quando necessário, inicia o segundo fator. A sessão autenticada só é aberta depois da conclusão dessas etapas.

19. Como as senhas são armazenadas?

Com `password_hash()`. A conferência é feita por `password_verify()`. Hash não é criptografia reversível: o sistema não recupera a senha original.

20. Como funciona o segundo fator?

O caminho mais forte utiliza TOTP, com códigos temporários de aplicativo autenticador. O segredo é protegido com AES-256-GCM. Existe também o caminho acadêmico por pergunta cadastral. Depois de três erros, o desafio é encerrado.

21. Qual segundo fator é mais seguro?

O TOTP é mais seguro. Perguntas sobre nome materno, nascimento e CEP foram mantidas como alternativa da especificação, mas essas informações podem ser conhecidas por terceiros.

22. Como o acesso por perfil é controlado?

As páginas restritas chamam `exigirLogin()` com os perfis aceitos. Ocultar itens do menu não é a proteção principal; o servidor também bloqueia quem digita diretamente um endereço que não pertence ao seu perfil.

23. Quais proteções de segurança foram implementadas?

Hash de senhas, consultas preparadas, tokens CSRF, escape contra XSS, controle de acesso por perfil, limite de tentativas, 2FA, cookies protegidos, renovação de sessão, logs, validação no servidor e isolamento entre empresas.

24. O que é CSRF?

É quando outro site tenta enviar uma ação usando a sessão da vítima. O sistema gera um token aleatório para os formulários e o confere no servidor antes de aceitar alterações.

25. O que é XSS e como foi tratado?

XSS ocorre quando uma entrada do usuário é exibida como HTML ou JavaScript. O projeto usa escape de saída com `htmlspecialchars()`, por meio da função `e()`.

26. Por que um agendamento cancelado não é apagado?

Porque o histórico é útil para relatórios, auditoria e atendimento. O registro permanece e muda para o status `cancelado`, guardando motivo e responsável. Os outros status são `agendado`, `confirmado` e `concluido`.

27. Como os horários são carregados sem atualizar a página?

O JavaScript chama endpoints da pasta `api/`, que devolvem JSON. A resposta serve para montar a interface; a validação definitiva é repetida no servidor na confirmação.

28. Como funciona o login com Google?

Usa OpenID Connect. O sistema guarda um `state` aleatório, envia o usuário ao Google, recebe um código, troca pelo `id_token` e usa o e-mail verificado para localizar a conta. O Google não cria sozinho o cadastro completo porque não fornece todos os dados exigidos.

29. Para que servem os logs?

Para registrar login, falha, segundo fator, bloqueio e logout. Nome e CPF ficam copiados no log para o histórico continuar compreensível mesmo se o usuário for excluído. Essa é uma desnormalização intencional.

30. O que acontece se uma integração externa falhar?

O projeto possui alternativas: endereço manual quando o ViaCEP falha; orientação quando e-mail não está configurado; fila ou envio manual para WhatsApp; e ocultação do botão Google quando as credenciais não existem.

31. Como o sistema foi testado?

Há testes de validação, autenticação, segurança, Google, multiestabelecimento, cadastros, lembretes, filiais e administração master. Quatro conjuntos independentes do banco principal foram executados nesta análise, totalizando 221 verificações aprovadas.

3. Perguntas críticas e respostas honestas

Quantas tabelas existem?

O arquivo `banco.sql` atual cria **26 tabelas**. O relatório informa 20 e a documentação do Supabase informa 25, portanto esses documentos precisam ser atualizados.

O sistema garante 100% que nunca haverá duas reservas simultâneas?

No MySQL/InnoDB, a transação e o bloqueio reduzem a condição de corrida. Para uma garantia uniforme em todos os bancos, especialmente PostgreSQL, seria adequado acrescentar um bloqueio específico por agenda ou uma restrição própria do banco. No PostgreSQL, `FOR UPDATE` não bloqueia uma linha que ainda não existe.

Por que a senha possui exatamente oito letras?

Essa regra veio da especificação acadêmica. Para produção, uma senha mais longa ou frase-senha seria mais segura. Também deve ser padronizada a recuperação de senha, que atualmente aceita uma regra diferente.

Existe alguma limitação no login Google?

A implementação verifica `state`, emissor, público, validade e e-mail confirmado. Uma implementação OpenID Connect mais rigorosa também deve validar criptograficamente a assinatura do `id_token`, usar `nonce` e considerar PKCE.

Qual melhoria você faria primeiro?

“Eu começaria padronizando as regras de senha e a documentação. Depois reforçaria a concorrência dos agendamentos no PostgreSQL e a validação completa do token do Google. Em seguida ampliaria os testes de integração e automatizaria sua execução.”

4. Checklist antes da apresentação

- Corrigir “20 tabelas” para **26** no relatório.
- Corrigir “25 tabelas” na documentação do Supabase.
- Atualizar o relatório para mencionar o suporte a PostgreSQL.
- Atualizar a descrição e a quantidade das suítes de teste.
- Atualizar o trecho que trata o WhatsApp automático como recurso futuro.
- Padronizar a regra de senha da recuperação.
- Saber explicar que a arquitetura é em camadas, mas não MVC formal.
- Não afirmar que um mecanismo é “100% inviolável”.
- Demonstrar o fluxo completo: login, serviço, filial, profissional, data, horário e confirmação.
- Ter dados de demonstração preparados antes de iniciar a apresentação.

O que você precisa dominar

1. Fluxo completo do agendamento.
2. Relacionamento entre as principais tabelas.
3. Interação entre página, model, PDO e banco.
4. Diferença entre validação no JavaScript e no servidor.
5. Transação e prevenção de conflito de horário.
6. Login, sessão, perfis e segundo fator.
7. Principais limitações e melhorias futuras.

> **Frase final sugerida:**
> “O principal diferencial técnico do projeto é que o horário exibido na interface não é tratado como garantia. No momento da confirmação, o servidor consulta novamente o banco, revalida as regras e só então grava o agendamento dentro de uma transação.”

Documento preparado a partir da implementação do projeto Agendei em setembro de 2026.